



Implementing Accountability

KATHY V. C. PARRIS, Lockheed Martin Tactical Aircraft Systems

The author shows how, through the use of an accountability model and its attendant management practices, a software development team can, by its own initiative, establish an interaction with its stakeholders through which it obtains resources, directions, assistance, and clarification of expectations.

In the prime of its life, the F-16 fighter aircraft is undergoing a heart transplant. A willing victim of its own success, the F-16 approaches midlife with customer demand for new and modified capabilities exceeding its already repeatedly expanded avionics computer capacity. Affordability and agility, the imperatives of the post-cold-war military, drove the United States Air Force and F-16 contractor Lockheed Martin Tactical Aircraft Systems to embrace a bold, far-sighted solution: development of a modular mission computer designed to increase computing capacity by an order of magnitude and reduce the cost and schedule of future software changes.

To tackle this challenging project, the USAF F-16 System Program Office and Lockheed Martin elected to form an Integrated Product Development¹ team. The participants thus faced a dual challenge: development of state-of-the-art avionics hardware and software, concurrent with the development of team culture and modes of operation. This article presents a case study of the resulting management systems and underlying mental models, written from my partisan viewpoint as their chief advocate and designer.



CONTEXT

The software products of the IPD effort comprise more than four million lines of code, more than a quarter of which must meet hard real-time deadlines in a flight environment. The platform in which the flight program will operate is also concurrently in development, including a new multiprocessor message-passing computer architec-

Our desire to operate as an "intrapreneurial," self-managed team initially met with resistance.

ture, several custom application-specific integrated-circuit chips, associated device drivers, and real-time extensions to the Ada run-time system. Other hardware and software elements also developed or implemented concurrently are the software engineering workstation network, the Ada compiler for the modular mission computer, software build and configuration management tools, simulation software and test equipment for debug and integration, and the interfaces for loading MMC software into the airplane.

Flight software capability requirements are perhaps the best understood and most stable aspect of the project, since the MMC is to duplicate the functions of a mature, familiar avionics system. New requirements have been a minor part of the initial development effort, although add-on capabilities are expected to expand the software development effort by another million or so lines of code.

The well-known capabilities, however, were to be implemented in an unfamiliar way. The imperatives of affordability and agility called for

"capability modular" software,² to be achieved by adopting the Ada language and object-oriented analysis and design methodology in place of the F-16 legacy of Jovial language and functional partitioning. This switch of language and methodology, together with the transition from mainframe terminals to personal workstations, constituted an extensive concurrent reengineering of the software factory.

The magnitude of the MMC undertaking, combined with the budget cutbacks and refocusing of the USAF's military mission caused by the collapse of Communism, left the project creeping along at a severely underfunded pace for its first two years. In the final proposal, the very best we could challenge ourselves to do—better than we had ever done—proved barely good enough to win a contract. Strong interest from the F-16 consortium's European partners and from other foreign customers ultimately saved the MMC from financial starvation—but, ironically, also locked in rigid need dates. The resulting software development schedule, featuring three incremental flight-test releases of roughly equal task size, known as tapes, was universally viewed as high risk. The first tape was a schedule concern because of the methodology learning curve, concurrent hardware development, and concurrent development of the software engineering environment. Many thought that the second tape, constrained to a still shorter schedule that significantly overlapped development of the first tape, had crossed the line from high schedule risk to "impossible," even before Tape 1 difficulties further compressed Tape 2's schedule. Thus, the cost and schedule commitments needed to keep the program alive significantly magnified the technical challenges.

The scope and magnitude of the MMC development effort called for a sizable team: more than 500 contributors at Lockheed Martin Tactical Aircraft Systems; dozens more at computer subcontractor Texas Instruments'

plant some 60 miles away, plus roughly a dozen government participants. Since not all LMTAS contributors are full-time participants, the equivalent staffing level is around 250, with about 80 percent involved in software tasks. This team of seemingly unwieldy proportions is made manageable through an internal structure consisting of eight product teams plus a chief engineer and a program manager and their staffs.

Our desire to operate as an "intrapreneurial," self-managed team initially met with resistance from some of the departments providing team members. A contractual requirement to comply with IPD concepts¹ and establish IPD teams made our choice irrefutable, though not easy to implement. We came from a matrix organization of strong functional departments interwoven with separately managed programs, with technical direction flowing primarily through the functional hierarchy. Participation in teams was familiar, in the context of each team member representing a functional department and making decisions through functional management's authority (and often after their review). However, the IPD concept of a team—pulling qualified individuals from their functional homes and empowering them as a cohesive interdisciplinary work unit to conduct business through its own separate channels of accountability and elevation—was a major culture change with companywide repercussions. Collocating 300 team members (240 in a dedicated MMC development building, 60 in the systems integration laboratory) was an important step, without which the accountability process described herein could not have succeeded in bringing about the culture change.

Although the ultimate degree of success of the MMC development program is yet to be realized, interim results exceeding customer and company expectations validate the team approach and our still-evolving management systems. The government has recognized excellent team performance



with a special award in excess of \$2.3 million. The high-risk first software increment (Tape 1) recovered schedule and was released to flight test one week early, with glowing reports from test pilots on its unexpected level of maturity. The second "impossible" software increment, Tape 2, was released on schedule for laboratory testing. Tape 3, the final increment, passed critical design review on schedule.

Not all the interim results are so rosy. Tape 2 is struggling through a seemingly endless morass of integration difficulties, as did Tape 1. Late discoveries of chip problems have further confounded hardware-software integration and affected hardware production schedules. Campaigns are underway to reduce MMC memory and throughput utilization to meet specified reserve requirements. These difficulties notwithstanding, experience to date indicates that the accountability process described in the following sections is playing an indispensable role in what will prove to be a successful program.

ACCOUNTABILITY MODEL

The accountability model described here and summarized in Figure 1 is more prescriptive than descriptive: it advocates how things *should* be, rather than asserting how things *are*. It lives behind the scenes as an aid to the design and troubleshooting of management systems. The implementation and practice of these management systems gradually molds reality in the shape of the model, as will be seen later.

The model views accountability as a *proactive continuous process*, not as a vague concept of obligation, nor as a reporting relationship on an organization chart. The participants in this process are a team and its stakeholders. "Team" means the collection of people sharing responsibility for a desired result, and "stakeholder" means anyone affected by the desired result or the means of

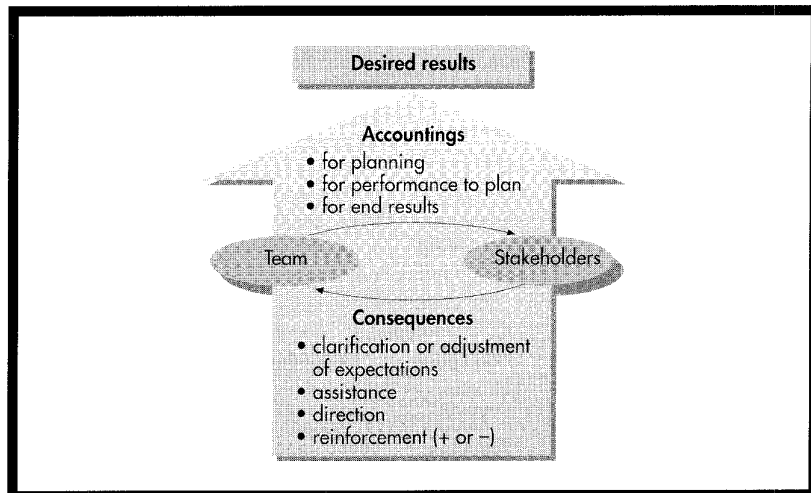


Figure 1. Our accountability model advocates how things should be, rather than asserting how things are.

achieving it. A team's stakeholders include its customers and suppliers, both internal and external; the functional organizations that provide people and other resources to the team; the members of the team; and any organizations of which the team is a part, such as larger program teams.

The process is an ongoing exchange of *accountings* and *consequences*. An accounting is a report of what you have done, are doing, or intend to do, backed up on request with an explanation of why the reported action makes sense. Consequences provide feedback, in the form of information, help, resources, restrictions, rewards, or punishment.

This exchange is intended to collectively determine *what makes sense* in a given situation, so that the team can then act accordingly. Every decision we make, individually or collectively, is determined by what makes sense to us at the time based on the information we have, the resources available to us, what we know how to do, and what we are motivated to do.³ Whenever a team's accountings make sense to its stakeholders, the team thereby establishes trust and "buy-in." When a team's account-

ings don't make sense to its stakeholders, open discussion of the information, resources, know-how, and motivators that influenced the team's decisions can often yield preferable alternatives.

An accounting may focus on end results, performance to plan, or planning. The end-results focus is traditionally the most common, yet the least effective, for if the results reported don't make sense to stakeholders, it is too late for alternatives to remedy the situation. The performance-to-plan focus is more productive, since the opportunity remains for correction if needed. Accountings for planning are by far the most powerful, because the best knowledge and resources of all stakeholders and team members together can usually identify weaknesses in a plan that the originator either couldn't recognize or didn't know how to correct. Furthermore, strengthening a plan up front is much more cost-effective than the recovery required if the plan fails. Unfortunately, the planning focus is also the least popular, verging on an infringement of privacy: "This is my responsibility; why should you meddle in my plans? I'm the one



accountable if you're not happy with the results."

The accountability cycle operates continuously throughout a project. The emphasis is likely to vary, focusing primarily on planning at the beginning, then shifting more toward performance-to-plan and end results. However, on a complex project, in which each stage reveals previously unknown information about the next stage, *planning never ends*, nor should accountability for planning. In a competitive environment where reduction of costs and cycle times is an ever-present goal, active accountability for planning continually fuels the improvement process.

We directly applied our accountability model to the design of management systems and team operating procedures. These systems and procedures were themselves means toward a higher end: shaping attitudes and behaviors, which are ultimately the greatest determinants of a team's success.

We directly applied our accountability model to the design of management systems.

Gradually, we learned to identify certain attitudes that resisted and conflicted with the empowering culture changes we sought, such as

- ♦ "Just tell me what you want, then leave me alone to get the job done."
- ♦ "The less information I share, the less can be used against me."
- ♦ "Needing help is a sign of weakness; receiving help is an admission of failure."

We wanted to replace *independent* behaviors with the habits of *interdependence*⁴; to get team members to think of "management" as an integral part of each team's responsibility rather than as

a privileged class of people; to emphasize *being* good rather than *looking* good. Early attempts to make this culture change through discussion evoked little disagreement, yet proved ineffective in changing behaviors. We realized that the systems already in place (not all of them conscious or visible) drove behaviors more forcefully than our advocated beliefs. By implementing new systems based on our accountability model, incrementally and with numerous corrections *en route*, we gradually began to see behaviors change.

IMPLEMENTATION

The management systems described here co-evolved over a period of two or more years. They are not clearly separable from each other, in description or in use.

Although we have hundreds of team members and even more stakeholders, none of these systems involves so many direct participants. A team leader or some other spokesperson appropriate to the occasion typically represents each of the eight product teams. Although we encourage the stakeholders who are most immediately affected to participate directly in the accountability process, those actively engaged are far outnumbered by, and serve as *de facto* proxies for, absentee stakeholders.

Team status reviews. One of the earliest implementations of the accountability process was our weekly, one-hour team status review. On a three-week rotation, each product team takes 10 to 40 minutes to give an accounting of performance to plan and end results. This is the primary formal channel for *routine* accountability—that is, for a glance at everything the team is doing, whether or not any special attention is warranted.

Each team tailors prescribed content guidelines to suit their activities and their stakeholders' information needs. TSR content includes activity roadmaps,

progress performance reports, and prioritized problem lists, as described later.

The program manager and chief engineer routinely attend TSRs, along with representatives of any teams dependent on the products or services of the presenting team. TSR sessions tend to be highly interactive. No matter how much dialogue has taken place outside this meeting, it is unusual to complete a TSR with no "course corrections" or realizations of some miscommunication.

To reinforce individual responsibility, personal action items are recorded with explicit closure criteria and suspense dates, and are tracked to completion. An action item is as likely to be assigned to a stakeholder as to a member of the presenting team, when the action required is removing some obstacle, clarifying requirements, reconciling conflicts, or providing missing information.

Each team is expected to take the initiative to identify risks and remove obstacles. For example, when presenting unmet resource needs, a team is expected to discuss not only what they need, but also what they have done to meet that need. Instances of whining, in which the team has not initiated appropriate action to fill their own needs, generally bounce back as an action item for the team leader. Instances where an obstacle duly elevated remains unresolved may create an action item for the chief engineer or program manager.

Activity roadmaps. Performance to plan would be easy to monitor if each team had only one activity, fully laid out according to one nice, linear plan. Unfortunately, this has never been the case for any of our product teams. Each team has multiple, overlapping, separate but often related activities, which seem to operate according to some vexatious law reminiscent of small children: *whichever one you're watching isn't the one getting into trouble*.

When we were right down to the line on the first major design review,

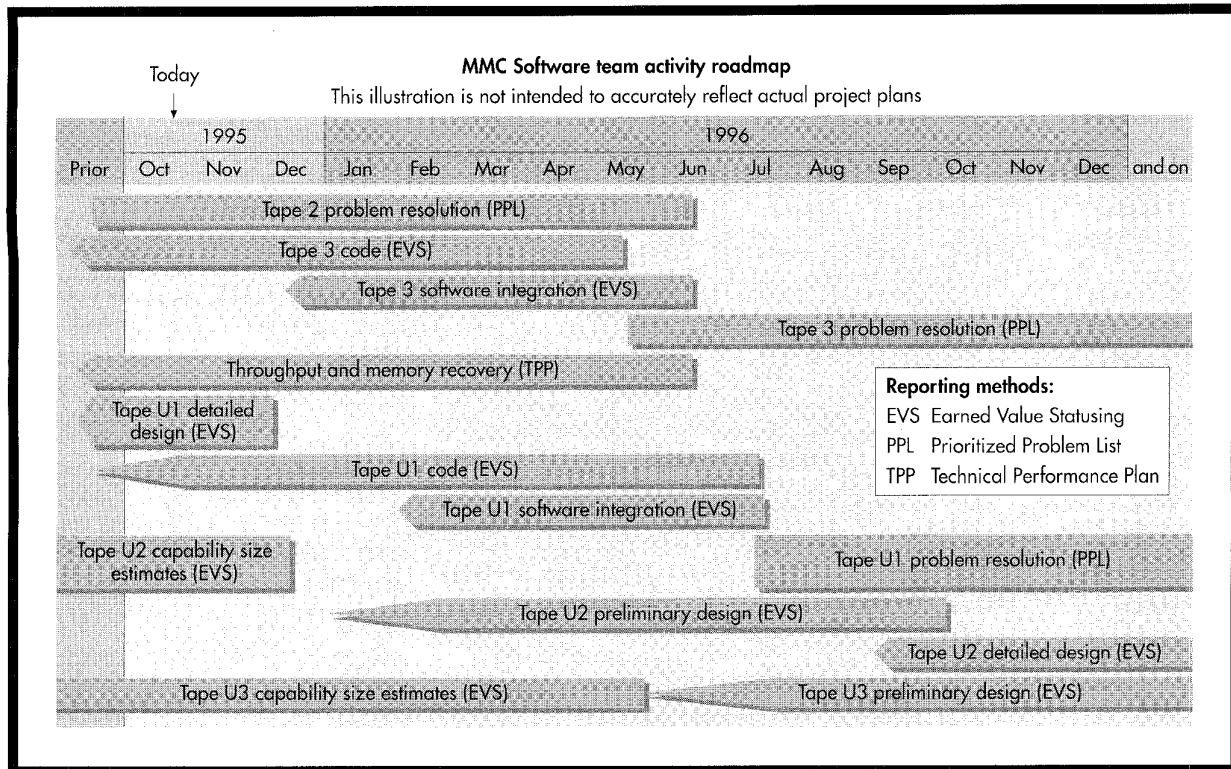


Figure 2. The activity roadmap. It includes, at a summary level, everything a team has to accomplish. Each bar represents a separate activity for monitoring purposes.

barely meeting a milestone already once postponed, we realized that the overlapping activity in preparation for the next major review had been overlooked in the rush. We were seriously behind before we started—which may have been inevitable, but at least we would have felt better had it been consciously reflected in our plans.

Rising from the flames of this incident (which singed the feet of a layer or two of management above the team) came the activity roadmap shown in Figure 2, fondly known as the “crayon chart” after the shape of the activity bars. Each team’s activity roadmap was to include, at a summary level, *everything* they had to accomplish. Each bar represented a separate activity for monitoring purposes. Unless agreed by stakeholders

and so noted on the activity bar, each activity was to have some kind of progress performance report to convey concisely yet accurately whether the activity was on track.

Recognizing that some downstream activities could not be planned in detail until certain prerequisite activities were completed, the points of the “crayons” were to indicate when detailed planning should be in place to guide ensuing activities. By glancing down the “today” line, one could instantly tell how many progress performance reports would together tell the whole story. Thus, in the TSRs, the activity roadmap served as an index to the progress performance reports that followed.

Once the team got used to the concept, the roadmap also became a valu-

able tool in detecting holes in the plan. A stakeholder concerned that a task may have been overlooked could be reassured by pointing to its activity bar, and, if necessary, unfolding the next layer of planning to pinpoint the task in question. If none of the “crayons” encompassed the task in question, the responsible team could be sent back to the drawing board to include it in their plans.

Progress performance reporting. To accurately track our planning and progress performance, we laid out time spans and dependencies for all major activities early in the program, using cost estimating models, corporate experience on comparable tasks, and critical-path methodology. Detailed task plans then evolved as the necessary information

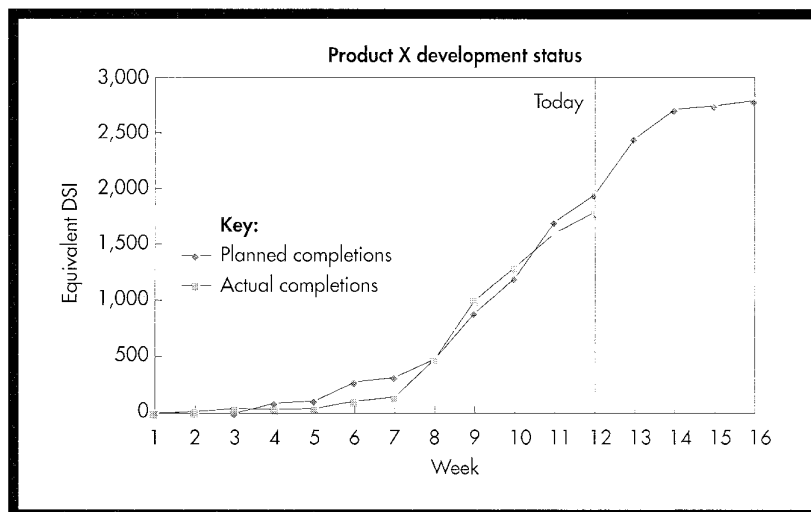


Figure 3. Sample earned-value summary chart. For software, work package size and the y-axis “earned value” are usually denominated in equivalent deliverable source instructions.

became available. If the point of a crayon passed the today line in evidence, we knew we were headed for trouble.

Some detailed task plans could be fully contained on a page or two of Gantt chart, in which case the progress report consisted of showing inchstones completed. Other detailed task plans comprised so many inchstones (sometimes thousands) that concise reporting necessitated statistical summaries. To serve this need, we inherited from the LMTAS Avionics Department the long-standing best practice of earned-value statusing.⁵

EVS is useful when many components must pass through the same series of process steps—for example, through requirements analysis, preliminary design, detailed design, coding, and so on. For each process step, explicit exit criteria are defined and a development phase weight is assigned to reflect what percent of the total process each step constitutes, in relative effort. Likewise, each component is assigned a work package size that reflects its proportion

of the total product. Detailed task schedules assign a due date for each component to complete each process step. When a component meets the exit criteria for a step, the actual completion date is recorded and earned value points are awarded, in the amount of work package size multiplied by development phase weight. Planned and actual cumulative values earned are plotted as a function of time along the x-axis to produce a concise statistical summary of performance to plan, as shown in Figure 3. For software, work package size and the y-axis “earned value” are usually denominated in equivalent Deliverable Source Instructions.

A stakeholder familiar with EVS charts can tell instantly from the slopes of the planned and actual lines, and the gap in between, when an activity is “headed south.” However, perfectly aligned planned and actual lines don’t always mean all is well. When using EVS to report progress, care must be taken not to let offsetting ahead-of-schedule and behind-schedule tasks paint a misleading picture. Two equally

weighted components or process steps may play significantly different roles in the critical path. In such cases, the reporting team supplements the EVS graph with back-up material—such as lower-level Gantt charts, exception lists, or explanatory notes—to support a determination of what, if any, corrective action is warranted.

Prioritized problem lists. Although the final product delivery is still well ahead of us, we have already “delivered” many internal products that are crucial to the completion of other products. Once the development of a product is complete, it is no longer covered by *progress* performance reporting. The focus shifts to how well the *product* is performing and how responsive the providing team is in resolving problems.

For each internally delivered product or product set, the providing team maintains a prioritized problem list. Although the list is maintained by the providing team, the prioritization is determined by the stakeholders. For instance, if an internally developed tool is used by several teams, the providing team collects tool-problem reports from all users, coordinates priorities with all affected teams, and uses the resulting prioritized list to work the most important problems first.

Although the addition, closure, and prioritization of problems happens on a day-to-day, person-to-person basis, a snapshot of the top five items on each list is also presented in TSRs. This “safety net” catches occasional lapses in communication. It also provides an opportunity for the more peripheral stakeholders to challenge whether the priorities really best serve program interests. For example, are the demands of a single user driving an inappropriate investment of program resources? Can some third party offer a workaround that is more affordable in the long run than the planned fix?

Special emphasis. Although the three-



week rotation of TSRs has been an effective method for routine accountability, many situations have arisen to necessitate a tighter accountability loop. When TSRs or informal communications indicate that an activity is "headed south" or already in trouble, it becomes a candidate for the program manager's special-emphasis list. A weekly meeting, chaired by the program manager, narrowly focuses stakeholder attention on the designated topics.

Here the responsible team presents a much narrower but deeper view of the at-risk or out-of-bounds activities. Emphasis is on determining consequences with an immediate positive effect: What can anyone do to improve performance-to-plan *now*? When warranted, the program manager calls for daily updates on the activity and related action items. Once identified for special emphasis, an activity remains on the weekly agenda until subsequent progress reports make it clear that the corrective action has been effective and there is no significant danger of relapse.

While fulfilling its intended purpose, this list has also generated some interesting side effects:

- ♦ *Extra motivation.* Although special emphasis is not intended to be punitive, a team may suddenly find itself able to cure a marginal problem through sheer determination to get off the list.

- ♦ *Organizational bonding.* Special emphasis is the only aspect of the accountability process in which the team members' home-department managers frequently participate. The team benefits from the managers' experience, perspective, and power to remove obstacles. The functional organization benefits by getting up-to-date information firsthand, thus avoiding unpleasant surprises. The team tends to have a healthier relationship of interdependence with the functional organizations whose managers do participate in the special-emphasis meetings, although it is unclear whether this is cause or effect.

- ♦ *Problem coordination.* Because of its

focus (problem resolution) and attendance (often broader than TSRs), it is not uncommon for a special-emphasis presentation to dissolve into on-the-spot troubleshooting or brainstorming of a possible solution. While a quick dose of this can render instant progress, discussions that ramble too far afield can soak up the valuable time of those waiting in line to discuss their own activities. Here the program manager as gatekeeper plays a real balancing act.

Handoffs. The accountability implementations discussed so far have a common prerequisite: you must first know who your stakeholders are and what they need. The needs of our ultimate customers were, for the most part, clearly identified in specifications and contract statements of work. The part that was hard to get our arms around was *what each team needed from each other*.

In the early planning stage, certain dependencies stood out clearly as we looked back from our still-imaginary end product: To deliver a validated avionics system, we needed MMC hardware, MMC software, and test equipment on which to verify it. MMC software in turn required an Ada compiler, an operating system, and many software development tools, not to mention a clear definition of software requirements and interfaces. It was what we *couldn't see* from this vantage point that concerned us.

In frustration, we abandoned our numerous attempts to draw out all the dependencies on paper. More and more dependencies became visible as detailed planning progressed, but often they were spotted with insufficient lead time. We needed an expeditious way to detect all the underwater hazards and incorporate them into our planning, or we would surely run aground.⁶

So we began the grueling process of defining *all* team-to-team handoffs. The thought process each team engaged in simultaneously went something like this:

1. What are all the outputs I must

provide, to an outside customer or to any other team?

- ♦ Who needs it?

- ♦ When will we provide it?

2. What are all the inputs I need from other teams to produce the outputs?

- ♦ Who will provide it?

- ♦ When do we need it?

3. For each output and input listed, where have we documented our provider/consumer agreements on

- ♦ What exactly will be provided (product requirements/capabilities)?

- ♦ What exactly will constitute completion of the handoff (terms and conditions)?

Using Filemaker Pro database software on a Macintosh computer, we developed an information model to capture and analyze the resulting answers. If a team identified the need for an input that no team planned to output, or if the provider's plans didn't meet the need date, the information model raised a flag and nagged us until we resolved the discrepancy. If an output was planned that nobody needed, after dou-

**More and more
dependencies
became visible
as detailed
planning
progressed.**

ble-checking for miscommunications, we deleted all associated tasks from plans. The question of "What other inputs do you need?" kept reasserting itself until all teams certified that their detailed planning had progressed far enough that they could confidently answer "None!"

During its first six months, 220 handoffs were incorporated into the information model, with another 10 percent trickling in since then. These



If giving good accountings is a hard skill to learn, dispensing valuable consequences is even harder.

figures exclude 60 handoffs that were recorded and later deleted for various reasons such as duplication, not needed, or represented an ongoing support activity rather than a handoff. No wonder our attempts to draw all the dependencies on a single sheet of paper had failed!

Basing it on user-friendly, flexible software made the information model easy to expand and revise to meet our evolving management needs. When we wanted to know all the handoffs that were necessary to get Tape 1 out the door, the handoff system could be tweaked to tell us. When we wanted to know everything that depended on a single handoff that was in jeopardy, a few keystrokes were all it took to find out. When we wanted to focus attention on just those handoffs that were past due or in jeopardy, it was easy to select them based on the latest status the providing team had given.

After the information model had paid its way by helping with macro planning, we unleashed it on micro planning. We were beginning to recognize some undesirable patterns, scenarios that seemed to get us in trouble over and over again:

- ◆ Provider and consumer did not reach agreement on exactly what would be handed off, or did not write it down precisely, resulting in the consumer not getting what he expected.

- ◆ Provider and consumer did not reach agreement on or did not record handoff terms and conditions, which resulted in the omission of essential tasks and affected the schedule. For example:

"What do you mean, it's delivered and on your dock waiting for me to pick it up? It's not delivered until it's installed in my lab!" or "What documentation? We didn't bid any."

- ◆ Provider's plans—task schedules, methods, staffing plans, resource plans, and so on—weren't thorough and robust enough to execute successfully.

- ◆ Provider lacked adequate means of apprising stakeholders of progress to plan.

To reflect each of these scenarios, fields were added to the information model to force inquiry about the adequacy of planning and to record the joint assessments of the providing team and its stakeholders. A checklist was devised to guide these adequacy assessments, for use by each team in developing their planning, and for continual update to reflect team learning. If an area of planning was deemed inadequate for some handoff, the information model demanded to know the next step: who would do what, by when, to improve the planning? The most serious cases required a long string of next steps before we were comfortable with the planning.

Early each week, each team lead receives a printout of all handoffs needing attention, and feeds the appropriate updates back into the information model through an administrator who verifies that all changes are consistent and have been properly coordinated. At midweek the team reviews any handoffs still flagged. This review is facilitated by an overhead system that displays the Macintosh screen to the entire audience, so that changes in the information-model can be made on-line as they are coordinated. Statistics of overall team performance on handoffs are also reviewed at this meeting, periodically resulting in challenges from the chief engineer to improve our batting average on meeting internal commitments.

Assistance. If giving good accountings

is a hard skill to learn, dispensing valuable consequences is even harder. Management "help" that doesn't help is such a familiar theme that countless engineers think Dilbert or his creator Scott Adams must live in the next cubicle. Yet this area offers some of the most notable success stories of the MMC experience.

We have found out what it's like to be far enough behind schedule to attract serious management attention. Such attention could easily have assumed team-unfriendly forms, such as replacing team leadership or dispersing the decision-making authority back into the functional departments. This didn't happen.

On the first such occasion, we clearly—in retrospect—needed help from outside the team, but hadn't asked for it. The help we received came in the form of the director of the affected functional department, who brought with him extensive software management experience and an inclination to reinforce rather than replace the team accountability process. Consequences during and as a result of his special assignment included

- ◆ recalibrating plans for the second increment of software development based on actual production rates for the first increment;

- ◆ shifting some highly visible customer design-review milestones to accommodate the more realistic time spans resulting from this recalibration;

- ◆ temporarily waiving the ceiling on overtime pay for team members with tasks on the critical path;

- ◆ adding experienced personnel, both from within LMTAS and from our partner, Texas Instruments;

- ◆ redistributing leadership responsibility *within* the team to ensure both planning and technical direction could receive adequate attention; and

- ◆ adding and upgrading workstations.

These examples of positive and constructive consequences came at a critical time in the team's growth.



Desirable behaviors and attitudes about assistance were further reinforced by an award-fee clause in the MMC contract. Each year the government determines what percent of a predetermined award fund best reflects exceptional contractor performance, based on criteria made known to the team at the beginning of the performance period. The criteria emphasized mutual assistance within the joint USAF/contractor team, and also the sharing of lessons learned with other projects. This emphasis inspired the following initiatives, reflected in a first-period award of over 94 percent:

- ♦ convening periodic "graybeard reviews" of highly experienced and respected industry and government experts to review our plans and status and make recommendations to the joint team,
- ♦ forming joint "red teams" to analyze alternatives and recommend solutions to program-critical problems,
- ♦ extracting applicable lessons learned from F-22 Advanced Tactical Fighter activities, and
- ♦ borrowing the most senior design and process experts available to help formulate key team decisions.

Customer accountability. USAF participants in the MMC program have operated quite effectively in a dual role, acting both as members of the joint MMC IPD Team and as the customer to whom the contractor team is accountable.

The most important and effective channel for customer accountability has been day-to-day, person-to-person communication. Because of the IPD approach, this ongoing communication is distributed among a dozen or more counterpart pairs, including but by no means restricted to the program manager and chief engineer.

To ensure these one-to-one channels don't calcify into silos, weekly teleconferences bring all the counterparts together for a review of open action

items, upcoming events, current status, and any problems or issues. The more formal channel of quarterly program management reviews brings the same players face-to-face for a thorough accounting of program performance-to-plan and results-to-date. These weekly and quarterly interactions are particularly important and effective for the ongoing clarification and occasional adjustment of customer expectations. They also afford us a socially acceptable opportunity to hold our customer accountable for meeting their commitments, such as action items, document approvals, and contract changes.

One of our hardest lessons to learn, in the sense of changing behaviors, has been in the area of customer accountability: the customer expects to hear the bad news as well as the good, and to share in the solution of problems before they become crises. The next section explains why this is such a hard lesson.

CONNECTIONS

Anyone seriously considering the possibility of implementing this kind of an accountability process should be aware of a couple of interdependencies: between accountability and trust, and between accountability and learning. Our experience indicates that neither of these interdependencies can be broken without rendering the accountability process ineffective. In choosing to implement our model, you marry the entire family.

Accountability and trust. Accountability depends on trust. The team offering up its accountings must have trust in its stakeholders:

- ♦ that consequences will be fair,
- ♦ that assistance will truly be helpful, and
- ♦ that direction will be knowledgeable and consistent.

Stakeholders must equally have trust in the team:

- ♦ that accountings will be honest,
- ♦ that no important information will be suppressed, and
- ♦ that the competing interests of multiple stakeholders will be fairly and appropriately balanced.

The bad news is that trust must be earned, not given. The good news is that a proactive accountability process is a powerful agent in bringing about trust. We have found that from a sufficient initial margin of trust, accountability and trust can and will symbiotically grow. We have also encountered instances where carefully planned interventions were required to reverse a downward spiral of distrust before an effective accountability process could be implemented.

Accountability and learning. Accountability depends on learning. More accurately stated, true accountability presumes a learning organization as described by Peter Senge in *The Fifth Discipline*.⁷ "Team learning," as Senge calls it, is what fuels the accountability cycle. The team must believe it can learn (that is, improve) from the feedback of stakeholders. Stakeholders must believe in their *obligation* to provide useful, carefully

**A proactive
accountability
process is a
powerful agent in
bringing about trust.**

considered consequences, as an integral part of the learning process. Stakeholders must also be willing to learn from the team's accountings and reflect their learning in ever more appropriate and effective consequences. Neither party can believe that learning or teaching is somebody else's job, or unnecessary.

Exercising accountability is a critical



Neither party can believe that learning or teaching is somebody else's job.

skill. There is an art to providing clear, comprehensive, yet concise accountings. There is wisdom in knowing when and how to help and when to refrain. Every team member and stakeholder should be part of an ongoing proficiency development program to cultivate such skills.

Learning to embrace a process that is based on unfamiliar mental models is difficult for team members. However, this learning process can be accelerated dramatically through consistent management reinforcement. In implementing our accountability process, we repeatedly found that notable learning bursts and measurable improvements in process adherence followed when the chief engineer or program manager focused personal attention on that process and reminded everyone of its importance.

Embracing new mental models is even harder for managers at the upper boundary of whatever organization is attempting the culture change. To begin with, if the next higher layer of management is outside the culture change boundary, there is no source of consistent management reinforcement to accelerate the learning process. Furthermore, changing mental models is truly an act of courage for those at the upper boundary. Raymond Smith, CEO of Atlantic Bell, describes this discomfort in an interview with *Business Ethics*⁸ magazine:

I had to completely change my own mindset. And that meant taking

someone who had learned skills exceedingly well, and in effect trading them in for a new, softer set. It's much easier to command and control than to be an empowering, creativity-encouraging boss. It means being exposed, to allow people to coach you.

MASTERING ACCOUNTABILITY

As instigator of the proactive accountability process, I look back and marvel at how far we've come; I look ahead and despair at how far away the vision still seems. We have a long way to go to master accountability; the MMC program may well end first. The pursuit will continue until it does. Here's a sampling of what's on the road ahead, ranging from near-term possibilities to wishful thinking.

Propagating mental models. To date, primarily the first and second tiers of team leadership—less than 10 percent of team members—have been directly involved in the accountability process. For the most part, changes in attitudes and behaviors are visible in those who have participated. The potential benefits of proactive accountability and the underlying mental models are the same regardless of the scope and level of team to which they are applied. We have yet to propagate what the leaders have learned throughout the remainder of the team.

Robust planning. Accountability for planning remains our greatest opportunity for improvement. For our next generation of accountability-for-planning systems, we are adapting ideas from the quality process.

Genechi Taguchi, one of the masters of the quality process, focuses on the concept of "robustness": the degree to which a product overcomes unplanned variations in its environment. He asserts that "product 'robustness' is more a

function of product design than of on-line control."⁹ By analogy, program robustness, the likelihood of achieving desired results in the presence of noise, should be more a function of planning than of performance-to-plan monitoring and consequences. The derived notion of "planning robustness" seems a promising foundation for a more proactive version of risk management. Viewed from this angle, program risk is just an inverse measure of planning robustness. Hence "risk management" means identifying noise (how reality may vary from assumptions), finding the nonrobust areas in plans, strengthening them if possible, and otherwise making allowances for the possible repercussions in coordination with stakeholders.

If the most productive strategy then is to detect the weak areas in plans as early as possible, why not apply proven techniques such as software inspections¹⁰ in the planning process? Our previously mentioned handoff-planning adequacy-assessment checklists provide a starting point for pursuing this strategy.

Just rewards. We have seen that the ability to proactively hold ourselves accountable—implementing management systems that in turn shape our mental models—lies largely within our sphere of control as a team. The further we progress, the more our ability to proceed is affected by factors that lie, at best, only marginally within our sphere of influence. Our success in mastering accountability depends on each individual's conviction that fulfilling multiple stakeholder expectations in a balanced fashion defines success. Yet this desired belief is often undermined by the reward systems that dominate not only our company but most of industry.

Even when bombarded with daily examples, it is hard to decipher what is happening to us here. We begin with well-considered corporate visions, missions, goals, and objectives. To base each individual's reward on his or her



contribution to these objectives, we dissect corporate goals and disperse chunks to line organizations, further subdividing the chunks at each level of the hierarchy. Then we define metrics so that we may objectively measure each chunk. Naturally, we reward the people within each line organization based on these metrics. So what's going wrong?

Optimizing the chunks does not optimize the organism from which they were sliced. Subdividing the original goal carves the life out of it. What is best for the organism is frequently suboptimal for some chunks, which all but guarantees opposition from their owners. Absent preventive measures, such opposition can kill a significant global cost reduction innovation that makes one organization's metrics look bad.

To make matters worse, the metric is only an *indicator* of the chunk, and not always a very good one. The map, however precise, is not the territory. So maximizing the metrics doesn't necessarily even optimize the chunks: the metric becomes the graven image sub-

stituted for the forgotten goal. A goal of delivering perfect products may lead us to base rewards on defect-density reductions. Absent preventive measures, defect density can and will be reduced by *not finding or reporting defects* if this is the behavior the reward reinforces.

The degree to which many team members are willing to divorce themselves from the reward system in preference for "doing the right thing" is truly remarkable, and adds credence to the theory that monetary rewards are a minor factor in employee motivation. Unfortunately, punishment is a stronger motivation, even when limited to subtle disapproval. Potential reward, when withheld, equates to punishment. Many who are willing to forego personal gain in the interest of furthering team goals will retreat when faced with the prospect of costing their boss a metrics-based reward.

We persist in this folly not because we're stupid, but because we have not yet found a way to design a fair and objective reward system that reinforces desirable values and behaviors.

Management by objectives is deeply rooted. Raymond Smith⁸ argues convincingly for moving beyond MBO to *management by commitment*, but the implementation of a corresponding reward system is not discussed in the interview cited. Does anyone out there have a turnkey *management by behaviors* reward system? The search continues.

Five years of MMC team experience substantiate some significant conclusions that, with two years remaining in the program, must still be qualified as "interim." MMC experience has shown that a team can, by its own initiative, establish an interaction with its stakeholders through which it obtains resources, directions, assistance, and clarification of expectations—in short, can empower itself. Management systems that are based on our accountability model, and in turn reinforce responsible attitudes and behaviors, are the key to this empowerment strategy. We hope to prove that this strategy can turn an "impossible" program into an unqualified success. ♦

REFERENCES

1. L.J. Menker, *Results of the Aeronautical Systems Division Critical Process Team on Integrated Product Development*, Tech. Report ASD-TR-90-5014, Air Force Systems Command, Wright-Patterson AFB, Ohio, 1990.
2. R.D. Schlappe and R.W. Royston, "Software Reuse for Capability Modularity," *Proc. 1995 AIAA Computing in Aerospace Conference*, accessible via <http://lmtas.com/CapMod/AIAAIndex.html>.
3. C. Carr, *Smart Training*, McGraw-Hill, New York, 1992, pp. 13-18.
4. S.R. Covey, *The Seven Habits of Highly Effective People*, Simon & Schuster, New York, 1989.
5. D.W. Benson, "An Earned-Value Methodology for Statusing Software Development," *Proc. STARS Business Practices Area Management Workshop*, Software Technology for Adaptable Reliable Systems Technology Center, Advanced Research Projects Agency, Arlington, Va., 1985; also accessible via http://lmtas.com/CapMod/ev_main.htm.
6. P. Conklin, "Enrollment Management, Managing the Alpha AXP Program," *Digital Technical J.*, Vol. 4, No. 4, Special Issue, 1992. Reprinted in *IEEE Software*, July 1996, pp. 53-64.
7. P.M. Senge, *The Fifth Discipline*, Doubleday/Currency, New York, 1990.
8. M. Kelly, "Interview with Raymond Smith," *Business Ethics*, May/June 1992, pp. 28-30.
9. G. Taguchi and D. Clausen, "Robust Quality," *Harvard Business Rev.*, Jan./Feb. 1990, pp. 65-75.
10. A.F. Ackerman, L.S. Buchwald, and F.H. Lewski, "Software Inspections: An Effective Verification Process," *IEEE Software*, May 1989, pp. 31-36.



Kathy Van Cleave Parris is a full-time leader of the Modular Mission Computer Team at Lockheed Martin Tactical Aircraft Systems. As an engineering project manager, her roles include team effectiveness facilitator, vision tender, management system designer, gadfly, listener, referee, wordsmith, web weaver, and learning advocate. Her 22 years in aerospace as software engineer, system designer, supervisor, chief engineer, instructor, team leader, and program manager include contributions to space shuttle, F-111, and F-16 avionics, simulation, and test systems.

Parris received a BA in Mathematics and Philosophy from Rice University.

Address questions about this article to Parris at MZ 5963, Lockheed Martin Tactical Aircraft Systems, P.O. Box 748, Fort Worth, TX, 76101; kvparris@lftc.lockheed.com.